Data Mining Assignment 1 - Report

Date: April 09, 2026
Subject: Data Mining (Assignment 1)
Author: Arthur Lenné, 314706802 **Github-Link:** https://github.com/axxtur/DM2026-Assignment-1

---

1. Linear Regression (Task 1 & 2)

1.1 Experimental Setup

The custom Linear Regression model was evaluated on four different datasets (A, B, C, D) using three learning rates: 0.1, 0.01, and 0.001.

1.2 Results (Grid Comparison)

Learning Rate: 0.1 Loss Curves:

Dataset A

Dataset B

Dataset C

Dataset D

Metrics:

Dataset A Metrics

Dataset B Metrics

Dataset C Metrics

Dataset D Metrics

Learning Rate: 0.01 Loss Curves:

Dataset A

Dataset B

Dataset C

Dataset D

Metrics:

Dataset A Metrics

Dataset B Metrics

Dataset C Metrics

Dataset D Metrics

Learning Rate: 0.001 Loss Curves:

Dataset A

Dataset B

Dataset C

Dataset D

Metrics:

Dataset A Metrics

Dataset B Metrics

Dataset C Metrics

Dataset D Metrics

1.3 Detailed Observations and Analysis

This section presents the evaluation of a linear regression model applied to four distinct datasets (labeled A, B, C, and D) across three different learning rates: 0.1, 0.01, and 0.001. The training process was conducted over a fixed duration of 500 epochs. To assess the model's performance, the Mean Squared Error (MSE), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R 2) were recorded.

An analysis of the loss curves reveals a strong dependency on the chosen learning rate regarding the model's convergence. For the learning rates of 0.1 and 0.01, the models successfully reached convergence within the allotted 500 epochs. With a learning rate of 0.1, the loss drops precipitously in the first few epochs, forming a sharp curve before plateauing. The learning rate of 0.01 exhibits a slightly smoother, more gradual descent but ultimately plateaus well before the 500-epoch mark. Because linear regression optimizes a strictly convex loss function with a single global minimum, both learning rates (0.1 and 0.01) eventually settle at the exact same optimal weights. Consequently, the final evaluation metrics for both configurations are virtually identical. In contrast, the learning rate of 0.001 proved to be too small for the given timeframe. The loss curves for this rate show a steady but slow decline, indicating that the model was still learning and had not yet converged when the training was terminated. This lack of convergence is quantitatively reflected in the significantly poorer metrics, including negative R^2 values for Datasets A and B, which indicate that the unconverged model performs worse than a simple horizontal line predicting the mean of the target variable.

Beyond the learning rates, the evaluation metrics highlight substantial differences in how well the linear regression model fits the individual datasets. The

model performs exceptionally well on Datasets C and D, achieving high R^2 values of approximately 0.995 and 0.972, respectively, once converged. This suggests a highly linear relationship between the features and the target variable in these datasets. Dataset A shows moderate predictability with an R^2 of 0.569, indicating that while a linear trend exists, there is a considerable amount of variance that the linear model cannot capture.

Dataset B presents a notable anomaly in both the loss curves and the final metrics. Even after full convergence (at learning rates 0.1 and 0.01), the R^2 value remains exceptionally low at approximately 0.045, accompanied by high error rates (MSE: 0.2137). This demonstrates that linear regression is fundamentally unsuitable for Dataset B, likely because the underlying data distribution is highly non-linear or consists largely of noise. Furthermore, the loss curves for Dataset B consistently show the validation loss settling at a lower value than the training loss. In the context of this experiment, this phenomenon most likely points to a specific data split where the validation set happens to be "easier" to predict than the training set—for instance, if the validation set contains less variance or fewer outliers. It could also be an artifact of regularization (if applied), which penalizes the training loss but is not added to the validation loss during evaluation.

Overall, the results underscore the importance of tuning the learning rate to ensure convergence within the available computational budget, while also demonstrating that the inherent linearity of the dataset dictates the absolute performance ceiling of a linear regression model.

---

2. Logistic Regression (Task 2 continued)

2.1 Decision Boundaries (Task 2a)

This section displays the learning curves and decision boundaries for all four datasets across different learning rates.

Learning Rate: 0.1 Dataset A

Dataset B

Dataset C

Dataset D

Learning Rate: 0.01 Dataset A

Dataset B

Dataset C

Dataset D

Learning Rate: 0.001 Dataset A

Dataset B

Dataset C

Dataset D

2.2 Confusion Matrices and Metrics (Task 2b)

This section provides the evaluation results for the logistic classifier, showcasing the separation quality for each dataset.

Learning Rate: 0.1 Dataset A

Dataset B

Dataset C

Dataset D

Learning Rate: 0.01 Dataset A

Dataset B

Dataset C

Dataset D

Learning Rate: 0.001 Dataset A

Dataset B

Dataset C

Dataset D

2.3 Observations

In this section, a logistic regression model was evaluated on four distinct datasets (A, B, C, and D) to perform binary classification. The training process was executed over 500 epochs using three different learning rates: 0.1, 0.01, and 0.001. To assess the classification performance, the models were evaluated using Accuracy, Precision, Recall, the F1-score, and Confusion Matrices.

The loss curves strongly illustrate the impact of the learning rate on model convergence. At a learning rate of 0.1, the training and validation loss curves exhibit a steep initial decline, rapidly flattening out to indicate successful and

fast convergence. A learning rate of 0.01 produces a smoother, more gradual descent. While it reaches a competitively low loss by the end of the 500 epochs, the slight variations in final metrics compared to the 0.1 learning rate suggest that the model might still be undergoing minute adjustments to its decision boundary. In stark contrast, a learning rate of 0.001 leads to severe underfitting. The loss curves for this rate are nearly linear and decline very slowly, proving that 500 epochs are entirely insufficient for the model to learn meaningful patterns at such a small step size.

The consequences of this underfitting at a learning rate of 0.001 are strikingly visible in the evaluation metrics. For Datasets A and B, the model achieves an accuracy of roughly 38.7% and 39.2%, respectively. In a binary classification context, an accuracy significantly below 50% is worse than random guessing. This indicates that the model was trapped near a poor random weight initialization and lacked the necessary step size to navigate toward a better decision boundary within the given timeframe. Conversely, Datasets C and D show relatively high accuracies even at this low learning rate (approx. 84% and 83%), suggesting that their optimal decision boundaries might have been closer to the initial state or are generally easier to approximate.

Analyzing the converged models (specifically at the optimal learning rate of 0.1) reveals clear differences in the linear separability of the four datasets. The logistic regression model achieves exceptional performance on Dataset C, with an accuracy of 97.62% and an F1-score of 0.9785. The corresponding confusion matrix shows very few misclassifications (13 false positives and 25 false negatives), demonstrating that the classes in Dataset C are almost perfectly linearly separable. Dataset D and Dataset A also show strong linear separability, yielding robust accuracies of 92.25% and 90.50%, respectively.

Dataset B, however, proves to be the most challenging for the logistic regression model. Even at full convergence (learning rate 0.1), the accuracy peaks at only 78.00%, with the confusion matrix showing a significant number of misclassifications across both classes (49 false positives, 39 false negatives). This relatively poor performance strongly suggests that the classes in Dataset B overlap considerably or require a non-linear decision boundary to be effectively separated. Notably, this difficulty aligns with the anomalies observed in the linear regression task, further confirming that Dataset B possesses complex or noisy underlying data structures that linear models struggle to capture.

2.4 Impact of Iterations (Epochs)

In this experiment, we kept the learning rate constant (at 0.01) and varied the number of iterations to observe how the decision boundary evolves as the model converges.

500 Iterations Dataset A

Dataset B

Dataset C

Dataset D

1000 Iterations Dataset A

Dataset B

Dataset C

Dataset D

1500 Iterations Decision Boundaries:

Dataset A

Dataset B

Dataset C

Dataset D

Confusion Matrices (Comparison by Epochs):

500 Epochs - Confusion Matrices Dataset A

Dataset B

Dataset C

Dataset D

1000 Epochs - Confusion Matrices Dataset A

Dataset B

Dataset C

Dataset D

1500 Epochs - Confusion Matrices Dataset A

Dataset B

Dataset C

Dataset D

2.5 Analysis

In this phase of the experiment, the impact of the training duration was analyzed by varying the number of iterations (epochs). The logistic regression model was trained for 500, 1000, and 1500 epochs while maintaining a constant

learning rate of 0.01 (as indicated by the implementation code, despite the notebook's markdown header). The progression of the loss curves, paired with the corresponding confusion matrices and evaluation metrics, provides clear insights into the model's convergence behavior.

At 500 iterations, the loss curves are still on a downward trajectory, suggesting that the model has not yet reached its optimal weights. This underfitting is quantitatively confirmed by the evaluation metrics. For instance, Dataset C achieves an accuracy of 94.56%, and Dataset A reaches 87.75%. While these are reasonable starting points, the subsequent training phases reveal that the model was prematurely stopped.

Extending the training to 1000 iterations yields a significant and measurable improvement across all four datasets. The loss curves begin to flatten asymptotically, indicating that the model is approaching the global minimum. This geometric convergence is directly reflected in the metrics: Dataset C experiences a substantial accuracy jump to 97.44%, with its confusion matrix showing a sharp decrease in false positives (from 25 down to 10) and false negatives (from 62 down to 31). Similarly, Dataset A's accuracy improves to 90.25%, and Dataset D rises to 92.63%. Even the inherently difficult Dataset B sees a slight performance bump from 76.25% to 77.75%.

Increasing the training duration further to 1500 iterations demonstrates the phenomenon of full convergence. Between 1000 and 1500 epochs, the loss curves become nearly horizontal. Crucially, the evaluation metrics plateau. The accuracy for Dataset A remains static at 90.25%, and Dataset B remains at 77.75%. Datasets C and D show only negligible microscopic fluctuations (Dataset C improves marginally to 97.88%, while Dataset D slightly fluctuates to 92.56%). The confusion matrices between 1000 and 1500 iterations are almost identical, often shifting by only one or two data points.

Furthermore, while the loss curves for Datasets C and D reveal a slight widening gap between training and validation loss towards 1500 epochs (often termed the generalization gap), the corresponding validation accuracies do not degrade. This confirms that the model is successfully generalizing and is not experiencing detrimental overfitting, but merely settling into its final decision boundary.

In conclusion, this temporal analysis proves that for a learning rate of 0.01, 500 epochs are insufficient for a logistic regression model to converge on these specific datasets. The optimal training duration lies around 1000 iterations, after which the model reaches its performance ceiling. Additionally, the prolonged training reaffirms the structural nature of Dataset B: even with abundant training time, the accuracy is hard-capped at approximately 78%, confirming that the dataset's classes are not strictly linearly separable and cannot be resolved simply by increasing the number of epochs.

---

3. Data Preprocessing (Task 3)

3.1 KNN Imputation Results

Comparing statistics before and after filling missing values with `KNNImputer` (k=5).

Preprocessing Stats 1

Preprocessing Stats 2

3.2 Observations

To address the missing data within the dataset, a K-Nearest Neighbors (KNN) imputation strategy was implemented. Using the KNNImputer class from the scikit-learn library, the model was configured to evaluate the five closest data points (n_neighbors=5). Within the preprocessing pipeline, feature columns were first converted to numeric formats, and the initial median and standard deviation for all columns containing NaN values were recorded. The KNN imputer was then applied to the feature space, effectively replacing the missing entries with the averaged values of their respective nearest neighbors.

A comparison of the statistical metrics before and after the imputation reveals a distinct pattern regarding the dataset's distribution. The standard deviation consistently decreased across all affected columns. For instance, the standard deviation of SepalLengthCm dropped from 1.037115 to 1.009275, and PetalLengthCm decreased from 1.582807 to 1.514955. This reduction in variance is an expected mathematical consequence of the KNN algorithm. Because the method fills missing entries using the localized average of nearby data points, it inherently smooths the dataset. It inserts values that are highly typical for their local neighborhood, which artificially clusters the data closer together and logically reduces the overall dispersion.

In contrast to the decreasing standard deviation, the median values remained highly stable. For SepalLengthCm (6.300000), SepalWidthCm (2.900000), and BranchLength (16.300000), the medians were perfectly preserved. Minor, negligible shifts were only observed in PetalLengthCm (shifting slightly from 5.085612 to 5.035683) and PetalWidthCm (from 1.600000 to 1.700000). This observation confirms that while the KNN imputation effectively minimizes the variance introduced by the missing data, it is highly robust at maintaining the central tendency. The overall shape and central distribution of the original features remain intact, providing a clean and reliable foundation for the subsequent classification tasks.

---

4. Data Exploration (Task 4)

4.1 Histogram of PetalWidthCm

Observation: While the histogram displays a prominent central cluster that peaks around 1.5, the most critical observation is the presence of isolated, anomalous data points at the extreme ends of the spectrum (specifically at values of -1, 0, and 4). Rather than indicating a true bimodal distribution, these isolated bins strongly suggest the presence of extreme outliers or potentially erroneously encoded missing values (e.g., a value of -1 might represent a measurement error). Identifying these anomalies is crucial, as they can heavily skew model training if not handled properly.

4.2 Pearson Correlation with PetalWidthCm

Largest Positive Correlation:

Top 5 Strongest Negative Correlations:

Observation on Correlation Strengths: `PetalWidthCompactness` exhibits a near-perfect positive linear relationship with the target column ($r \approx 0.9917$). In stark contrast, while the bottom 5 features mathematically represent the "strongest" negative correlations, their actual values (ranging from -0.074 to -0.096) are exceedingly close to zero. This indicates that there is virtually no meaningful inverse linear relationship between any feature in the dataset and `PetalWidthCm`.

4.3 Boxplot Analysis

Observation: The boxplots provide a clear visual comparison of the underlying distributions of these highly correlated features, revealing distinct structural differences. `SepalGlossIndex` shows a notably wider variance (Interquartile Range) centered around 0. Interestingly, the four negatively correlated features (`SepalWidthMajorAxis`, `SepalWidthCompactness`, `SepalWidthCurvature`, `SepalWidthMinorAxis`) exhibit nearly identical, highly compressed distributions. They are tightly clustered with medians around 3.0, feature very narrow interquartile ranges, and display numerous extreme outliers on both the upper and lower whiskers. This striking similarity strongly suggests that these four features might be highly redundant or derived from the same base metric. Furthermore, the prevalent outliers across almost all plotted features emphasize the necessity for robust, outlier-resistant classification models in the next steps.^

---

5. Regularization (Task 5)

5(a) L2 Regularization: Loss Curves

To analyze the effect of L2 regularization during training, the model was trained with four different λ (lambda) values. The loss curves below illustrate the training and validation loss over 10,000 iterations for each setting.

(1) No Regularization ($\lambda = 0$)

(2) L2 Regularization ($\lambda = 0.01$)

(3) L2 Regularization ($\lambda = 1$)

(4) L2 Regularization ($\lambda = 100$)

Observation on Loss Curves: An analysis of the loss curves across the four different λ values reveals the profound impact of L2 regularization on model training. For the baseline model with no regularization (λ=0) and the model with a very weak penalty (λ=0.01), the loss curves appear virtually identical. Both exhibit a noticeable generalization gap, where the validation loss remains consistently higher than the training loss, indicating a slight tendency to overfit the training data. The penalty of 0.01 is mathematically too small to meaningfully constrain the model's weights. However, when the regularization parameter is optimally increased to λ=1, the generalization gap visibly narrows. The L2 penalty successfully restricts the weights, preventing the model from overadapting to the underlying noise in the training data and leading to a much more stable validation loss. Conversely, applying an excessive penalty of λ=100 restricts the weights so severely that the model loses its capacity to learn entirely. In this scenario, the loss drops only slightly during the initial iterations before flatlining at a very high error rate, demonstrating a classic symptom of severe underfitting.

5(b) Testing Results: No Regularization vs. L2 Regularization

Based on the loss curve analysis, a lambda value of 1 proved to be the optimal regularization parameter. Below is the direct performance comparison between the unregularized baseline model and the optimally regularized L2 model ($\lambda = 1$) on the testing set.

No Regularization ($\lambda = 0$)

L2 Regularization ($\lambda = 1$)

Performance Summary Table

Model Setting	Accuracy	Precision	Recall	F1-Score
No Regularization ($\lambda = 0$)	0.7200	0.7125	0.7500	0.7308
L2 Regularization ($\lambda = 1$)	0.7400	0.7342	0.7632	0.7484

Observation and Analysis:

Based on the insights gained from the loss curves, a λ value of 1 was identified as the optimal regularization parameter. A direct comparison of the testing results between the unregularized baseline model and this optimally regularized model demonstrates a clear, measurable improvement in predictive performance. By penalizing large weights, the L2 regularized model becomes less sensitive to noise in the training data, which directly translates to better generalization on the unseen test set. Quantitatively, the overall accuracy improves from 72.00% to 74.00%, and the F1-score, which provides a balanced measure of precision and recall, increases from 0.7308 to 0.7484. Furthermore, the confusion matrix for the regularized model reflects this enhancement by showing a reduction in both false positives and false negatives compared to the baseline. It is also worth noting that while testing was conducted for the extreme case of λ=100, the model had completely collapsed into a trivial state—predicting only a single class and yielding an accuracy of roughly 50.67%. This total collapse further confirms that λ=1 strikes the correct balance between bias and variance for this specific classification task.